

Operating Systems

Assignment 1

Character Device Driver

NAME : K. D. Karun Lakshman
ROLL NUMBER : B240213CS
COURSE : Operating Systems(CS2012E)

1. Problem Statement

The objective of this assignment is to design and implement a Linux Character Device Driver as a loadable kernel module. The driver accepts two parameters at insertion time: **kernel_version**, an integer array specifying the major and minor version of the kernel, and **timer**, the number of seconds within which two actions must be completed.

Upon insertion, the module validates that the supplied kernel version matches the version against which the module was compiled, using the compile-time constant **LINUX_VERSION_CODE**. If they do not match, the module refuses to load. On successful insertion, the driver registers a character device and prints the allocated major number, minor number, and timer value to the kernel message buffer (viewable via `dmesg`).

After insertion, the user must perform two actions in strict order within the given time: (1) read from the device using `cat` or a similar tool, and (2) write a username to the device using `echo` or `write`. A kernel waitqueue is used to block the exit function until both actions are completed or the timer expires. When `rmmod` is called, the driver inspects whether both actions were completed in order and within time, then prints either a success message (including the username) or an appropriate failure message to the kernel log.

2. Methodology

2.1 Module Parameter Handling

Two module parameters are declared using the **module_param** family of macros provided by *linux/module.h*. The `kernel_version` array is handled by **module_param_array**, which populates an integer array and stores the count of supplied values in `kv_argc`. This count is checked to be exactly 2 before any version comparison is attempted, preventing out-of-bounds access. The timer value is a simple integer handled by **module_param**.

2.2 Kernel Version Validation

The macro **KERNEL_VERSION(major, minor, patch)** packs three integers into a single 32-bit value as $(major \ll 16) \mid (minor \ll 8) \mid patch$. The compile-time constant **LINUX_VERSION_CODE** holds this packed value for the kernel headers used during compilation. To compare only major and minor while ignoring the patch level, both values are masked with `& ~0xFF` before comparison. A mismatch causes `module_init` to return **-EPERM**, which makes `insmod` fail immediately.

2.3 Character Device Registration

The function **register_chrdev(0, DEVICE_NAME, &fops)** is called with 0 as the major number, instructing the kernel to allocate a free major number dynamically. The returned value is stored and later passed to **unregister_chrdev** in the exit function. A **file_operations** struct maps the `.open`, `.release`, `.read`, and `.write` function pointers to the driver's handler implementations. The kernel uses this table to dispatch all system calls made on the device file.

2.4 Read and Write Handlers

Kernel and userspace processes operate in separate virtual address spaces. Direct memory access across this boundary (e.g. using `memcpy`) would cause a kernel panic. The helper **`copy_to_user`** is used in the read handler and **`copy_from_user`** in the write handler; both validate the userspace pointer and handle page faults safely. The read handler sets the flag `read_done = 1`. The write handler only sets `write_done = 1` and calls **`wake_up()`** if read has already occurred, enforcing the required ordering.

2.5 Waitqueue and Timer

A waitqueue is declared statically using **`DECLARE_WAIT_QUEUE_HEAD(wq)`**. The timer is set up with **`timer_setup`** and armed with **`mod_timer`** using `jiffies + timer_val * HZ` as the expiry, where HZ is the number of kernel timer ticks per second. When the timer fires, its callback sets `timed_out = 1` and calls **`wake_up(&wq)`**. The exit function calls **`wait_event_timeout`**, which sleeps on the waitqueue until either both actions are complete or the timer fires, whichever comes first.

2.6 rmmod Validation Logic

After **`wait_event_timeout`** returns, **`del_timer_sync`** cancels any pending timer safely. The four possible outcomes are then evaluated in order: (1) both flags set and not timed out — success, print username; (2) `read_done` is 0 — failure, read never performed; (3) `write_done` is 0 — failure, write never performed; (4) `timed_out` is set — failure, actions not completed in time. The verdict is written to the kernel log via `printk` and is visible with `dmesg`.

3. Explanation of Key Kernel Concepts

`__init` and `__exit` macros

The `__init` attribute tells the kernel that the tagged function is only needed during module loading. Once loading is complete the kernel frees the memory occupied by that function, since it will never be called again. Similarly, `__exit` marks the cleanup function, which is discarded entirely for modules built into the kernel (since they can never be unloaded).

`printk` and the kernel ring buffer

`printk` is the kernel counterpart of `printf`. It writes formatted messages to the kernel ring buffer, which is a fixed-size circular log accessible via the `dmesg` command. Unlike `printf`, `printk` has no userspace context and cannot block, making it safe to call from interrupt handlers and timer callbacks. Log levels such as `KERN_INFO` and `KERN_ERR` control message priority.

`copy_to_user` and `copy_from_user`

These functions are the only safe way to transfer data across the kernel/userspace boundary. They validate that the supplied userspace pointer is within a legal userspace range, handle page faults that may occur if the target page is not yet in physical memory, and return the number of bytes that could not be copied (0 on full success). Using a raw `memcpy` across this boundary bypasses all these checks and will cause a kernel oops or silent data corruption.

Waitqueues

A waitqueue is a kernel data structure — essentially a list of sleeping processes or kernel threads — that enables event-driven synchronisation. **wait_event_timeout(wq, condition, timeout)** puts the caller to sleep on the queue until the condition expression evaluates to true or the timeout in jiffies elapses. **wake_up(&wq)** re-evaluates the condition for all sleepers and wakes those whose condition is now true.

jiffies and HZ

jiffies is a kernel global that increments once per timer tick. **HZ** is the number of ticks per second, typically 250 or 1000 depending on the kernel configuration. Therefore $jiffies + N * HZ$ represents a point N seconds in the future, which is the standard idiom for setting a timer expiry.

4. Execution Procedure

```
# 1. Build the module

make

# 2. Insert — replace 6,5 with your actual kernel major.minor

sudo insmod b240213cs_A1.ko kernel_version=6,5 timer=30

# 3. Check dmesg for the allocated major number

dmesg | tail -10

# 4. Create device node using the major number from dmesg

sudo mknod /dev/mydev c 0

sudo chmod 666 /dev/mydev

# 5. Perform READ action

cat /dev/mydev

# 6. Perform WRITE action (username written to device)

echo $(whoami) > /dev/mydev

# 7. Remove module and see verdict

sudo rmmod b240213cs_A1

dmesg | tail -10
```

Failure case — let the timer expire:

```
sudo insmod b240213cs_A1.ko kernel_version=6,5 timer=10 #

wait 10 seconds without performing any actions

sudo rmmod b240213cs_A1

dmesg | tail -5
```

Version mismatch case:

```
sudo insmod b240213cs_A1.ko kernel_version=5,2 timer=30

# insmod exits with error; check dmesg for rejection message

dmesg | tail -3
```

5. Screenshots

5.1 Successful Module Insertion

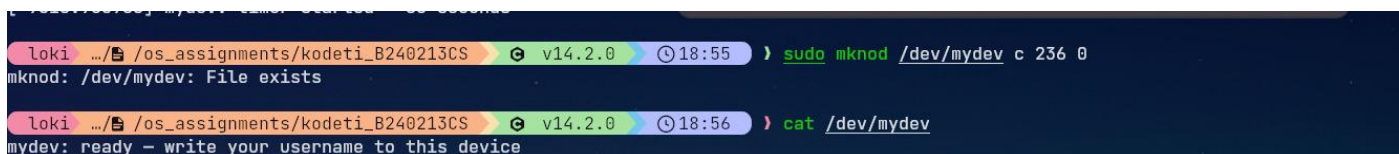
The screenshot below shows the terminal command used to insert the module and the dmesg output confirming the allocated major number, minor number, and timer value.



A terminal window showing the command `sudo insmod b240213cs_A1.ko kernel_version=6,12 timer_val=60` being executed. The prompt is `loki` in a directory `/os_assignments/kodeti_B240213CS` on a system with kernel `v14.2.0`. The time is `18:55`. Below the command, the output of `sudo dmesg | tail -3` is shown, displaying three lines of kernel messages: `[ 9616.950984] mydev: registered - major=236 minor=0 timer=60`, `[ 9616.950986] mydev: run: sudo mknod /dev/mydev c 236 0`, and `[ 9616.950988] mydev: timer started - 60 seconds`.

5.2 Device Node Creation and Read Action

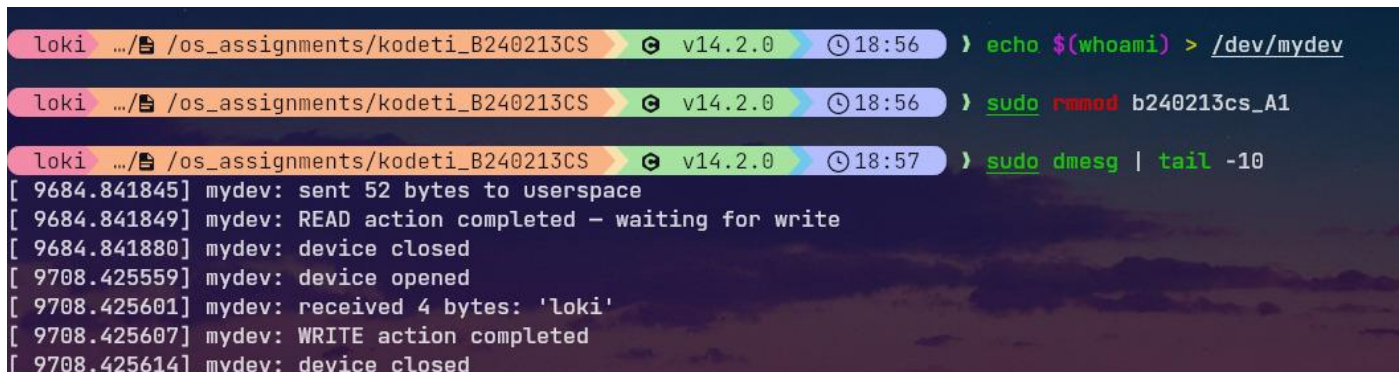
The screenshot below shows the `mknod` command creating `/dev/mydev` and the `cat` command triggering the read action. The dmesg output confirms 'READ action completed'.



A terminal window showing the command `sudo mknod /dev/mydev c 236 0` being executed. The prompt is `loki` in a directory `/os_assignments/kodeti_B240213CS` on a system with kernel `v14.2.0`. The time is `18:55`. Below the command, the output of `cat /dev/mydev` is shown, displaying the message `mydev: ready - write your username to this device`.

5.3 Write Action — Username Written to Device

The screenshot below shows the `echo` command writing the username to the device, and the dmesg confirmation that the write action was received and accepted.



A terminal window showing the command `echo $(whoami) > /dev/mydev` being executed. The prompt is `loki` in a directory `/os_assignments/kodeti_B240213CS` on a system with kernel `v14.2.0`. The time is `18:56`. Below the command, the output of `sudo rmmod b240213cs_A1` is shown, displaying the message `mydev: ready - write your username to this device`. Below that, the output of `sudo dmesg | tail -10` is shown, displaying ten lines of kernel messages: `[ 9684.841845] mydev: sent 52 bytes to userspace`, `[ 9684.841849] mydev: READ action completed - waiting for write`, `[ 9684.841880] mydev: device closed`, `[ 9708.425559] mydev: device opened`, `[ 9708.425601] mydev: received 4 bytes: 'loki'`, `[ 9708.425607] mydev: WRITE action completed`, and `[ 9708.425614] mydev: device closed`.

5.4 Successful rmmod — Success Message with Username

The screenshot below shows the rmmod command and the dmesg output printing the success message along with the captured username.

```
[ 9413.423133] mydev: device closed
[ 9413.426906] mydev: device opened
[ 9413.426925] mydev: received 4 bytes: 'loki'
[ 9413.426927] mydev: WRITE action completed
[ 9413.426930] mydev: device closed
[ 9413.444877] mydev: rmmod called - checking completion...
[ 9413.444884] mydev: Successfully completed the actions within time. Username: loki
[ 9413.444888] mydev: unloaded
```

5.5 Failure Case — Timer Expired

The screenshot below shows the failure scenario where the timer expired before both actions were completed. The dmesg output shows the timer expiry message and the failure verdict.

```
loki .../os_assignments/kodeti_B240213CS v14.2.0 18:57 sudo dmesg | tail -10
[ 9684.841845] mydev: sent 52 bytes to userspace
[ 9684.841849] mydev: READ action completed - waiting for write
[ 9684.841880] mydev: device closed
[ 9708.425559] mydev: device opened
[ 9708.425601] mydev: received 4 bytes: 'loki'
[ 9708.425607] mydev: WRITE action completed
[ 9708.425614] mydev: device closed
[ 9733.862619] mydev: rmmod called - checking completion...
[ 9733.862626] mydev: Failure - actions not completed within 60 seconds
[ 9733.862634] mydev: unloaded
```

5.6 Version Mismatch Rejection

The screenshot below shows insmod being rejected when a mismatched kernel_version is supplied. The shell reports an error and dmesg shows the version mismatch message from the driver.

```
loki .../os_assignments/kodeti_B240213CS v14.2.0 15:20 sudo insmod b240213cs_A1.ko kernel_version=5,2 timer_val=60
insmod: ERROR: could not insert module b240213cs_A1.ko: Operation not permitted
loki .../os_assignments/kodeti_B240213CS v14.2.0 15:20 sudo dmesg | tail -3
[ 293.264266] audit: type=1400 audit(1774777680.753:128): apparmor="DENIED" operation="open" class="file" profile="/usr/bin/evince" name="/usr/local/lib/libxkbcommon.so.0.0.13.1" pid=7703 comm="evince" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 414.168610] mydev: loading - timer=60 seconds
[ 414.168618] mydev: version mismatch! built for 6.12, got 5.2
loki .../os_assignments/kodeti_B240213CS v14.2.0 15:20
```

6. References

- [1] EmbeTronicX Linux Device Driver Tutorials — <https://embetronicx.com/linux-device-driver-tutorials/>
- [2] Apriorit — Simple Driver for Linux OS — <https://www.apriorit.com/dev-blog/195-simple-driver-for-linux-os>
- [3] Linux Kernel Documentation — kernel/module.h, linux/wait.h, linux/timer.h